

iOS Doll App Research Brief

Project Summary

This document captures the research and recommendations for building an iOS app that lets users create a customizable virtual doll, place it inside a stylized music-box scene, add decorative elements and music, render a short video, and share it externally.

The intended product flow is: 1. Animated landing screen 2. Step-by-step doll builder 3. Global constrained adjustment mode 4. Scene decoration and styling 5. Music selection 6. Local video render/export 7. Share flow 8. Paywall after the first successful share/export

Recommended Core Tech Stack

iOS App

- **Swift**
- **SwiftUI** for app UI and navigation
- **RealityKit** for 3D composition, scene assembly, and rendering
- **AVFoundation** for local video composition and export
- **StoreKit 2** for purchases and subscriptions
- **URLSession** for API and asset delivery
- **BackgroundTasks** for deferred sync/download work
- **Native iOS share sheet** for export/share

Backend

- **Golang** for API and backend services
- **Redis** for cache, queues, rate limits, hot state, and short-lived session data
- **PostgreSQL** for durable business data
- **S3-compatible object storage** for 3D assets, previews, and exported media
- **CDN** for fast asset delivery
- **Docker** for packaging
- **GitHub Actions** or equivalent for CI/CD
- **OpenTelemetry** for observability

Recommended Architecture Style

- Start with a **modular monolith** in Go
- Do **not** start with microservices
- Split into internal modules for:
 - auth
 - catalog
 - projects/compositions

- entitlements
 - analytics
 - admin/content operations
 - asset manifests
-

Product Concept

The app should let users assemble a doll from predefined categories of parts, rather than offering full freeform 3D modeling.

This is the right product approach because it allows: - controlled creativity - clean UX - performance predictability - easier asset validation - fewer broken combinations - faster iteration of content packs

The system should be based on **typed slots** with constraints.

Suggested Doll Slots

- head
 - hair
 - hat
 - horns
 - halo
 - body shell
 - inner insert
 - collar
 - wings
 - left hand
 - right hand
 - left sleeve
 - right sleeve
 - lower body
 - feet/base
 - tail
 - optional decorative pieces
 - background
 - foreground
 - floating props
 - music track
-

Customization Model

Each slot should support controlled customization through metadata.

Per-slot metadata should define:

- allowed asset IDs
- default transform
- min/max scale
- min/max rotation
- anchor point
- collision bounds
- dependencies
- exclusions
- optional animation compatibility flags

Design Principle

Users should be able to customize extensively, but only inside a controlled aesthetic and technical envelope.

That means: - no unrestricted freeform placement for core body parts - asset snapping to anchors - bounded transforms - compatibility rules between parts - visual consistency preserved by design

Backend Responsibilities

The Go backend should provide: - authentication - guest session handling - account upgrade flow - catalog delivery - asset manifest delivery - project save/load - versioned project persistence - entitlement verification - analytics ingestion - feature flags / remote config - admin moderation and publishing tools

PostgreSQL should store:

- users
- guest-to-account mappings
- doll projects
- project versions
- catalog metadata
- slot compatibility rules
- purchases and entitlements
- share history
- analytics facts that need durability
- experiment assignments
- admin content records

Redis should store:

- hot catalog/manifests
- rate limits
- feature flag cache
- temporary locks
- job state

- export/render state
 - short-lived project/session state
 - counters and ephemeral analytics buffers
-

3D Asset Pipeline Requirements

The source 3D scans must go through a production asset pipeline before they are usable in the app.

Required steps

- mesh cleanup
- retopology or simplification
- UV cleanup
- texture optimization/compression
- pivot/origin normalization
- scale normalization
- anchor placement
- collision mesh generation
- LOD generation if needed
- preview thumbnail generation
- metadata generation
- versioning

Each asset should include metadata for:

- asset ID
 - slot/category
 - display name
 - version
 - preview image URL
 - 3D file URL
 - texture references
 - bounding box
 - default transform
 - allowed transform range
 - compatible slots/assets
 - excluded assets
 - dependency rules
 - animation support flags
-

Rendering Strategy

Recommended V1

Use **on-device rendering** first.

Benefits: - lower infrastructure cost - faster turnaround for the user - fewer backend bottlenecks - simpler MVP architecture

Defer server-side rendering until:

- higher-resolution premium exports are needed
- videos become longer or heavier
- older devices struggle
- premium visual effects are required
- you need deterministic identical renders across devices

Recommendation

- **V1:** local render/export on device
 - **Later:** optional premium cloud render
-

Sharing Strategy

Treat Instagram as one important share destination, not the only publishing model.

Recommended approach

- generate the video locally
- open the native iOS share sheet
- let the user share wherever supported

This keeps the product flexible and avoids over-coupling the app to a single distribution channel.

Monetization Strategy

Recommended model

- free first successful export/share
- paywall after first successful export/share
- paid unlock for:
 - unlimited exports
 - premium asset packs
 - premium music
 - premium scenes/backgrounds
 - optional HD render

Payments stack

- StoreKit 2 on device
- server-side entitlement sync/verification in backend

- restore purchase support
-

Major Product Requirements

- guest onboarding
 - fast entry into creation flow
 - animated landing experience
 - step-by-step builder
 - optional skip for every slot
 - constrained global adjustment mode
 - scene styling mode
 - music selection
 - render/export flow
 - share flow
 - paywall trigger after first successful export/share
 - save draft projects
 - restore purchases
 - admin-controlled catalog updates
-

Major Technical Requirements

- slot-based composition engine
 - versioned asset manifests
 - project save/load model
 - offline-tolerant editing after asset download
 - autosave and crash recovery
 - analytics across the full funnel
 - entitlement caching
 - CDN-backed asset delivery
 - feature flags
 - remote config
 - moderation/admin controls
 - observability across app and backend
-

Non-Functional Requirements

- fast startup and time-to-first-edit
- predictable rendering performance on mid-range iPhones
- bounded memory usage
- resumable asset downloads
- backward compatibility for older saved projects
- stable export pipeline
- App Store compliance

- privacy-aware analytics implementation
 - scalable content publishing workflow
-

Major Decisions Still Needed

1. RealityKit-only or hybrid rendering stack

2. Recommendation: RealityKit-first

3. On-device vs cloud rendering

4. Recommendation: on-device for V1

5. Anonymous-first or account-first onboarding

6. Recommendation: anonymous-first, upgrade later

7. How much transform freedom to allow

8. Recommendation: constrained transforms per slot

9. One core doll schema or multiple doll template families

10. Recommendation: one core schema with optional families later

11. How many assets ship in the app bundle vs remote download

12. Recommendation: bundle only starter assets, deliver most remotely

13. Monetization model detail

14. Recommendation: first free export, then subscription or unlock + packs

15. Admin tooling scope

16. Recommendation: lightweight internal CMS/dashboard from early stage

17. Analytics depth

18. Recommendation: instrument every major step and asset interaction

19. Backend scale strategy

20. Recommendation: modular monolith first, split only when needed

Suggested Phased Roadmap

Phase 1: Prototype

- one doll template
- 3 to 5 slots
- 20 to 30 assets
- simple transform constraints
- one scene
- one music track
- local render/export
- native share flow
- simple paywall trigger

Phase 2: MVP

- full slot taxonomy
- remote catalog delivery
- saved projects
- analytics
- StoreKit 2 integration
- entitlement backend
- admin content pipeline
- crash/memory optimization
- experiment hooks

Phase 3: Scale

- premium asset packs
- seasonal content drops
- more scene templates
- optional cloud render
- onboarding/paywall experiments
- social/gallery layer if desired

Risks to Plan For

- 3D scan quality may be inconsistent and require significant cleanup
 - too much customization freedom can break visual quality
 - mobile memory usage can spike with large textures/meshes
 - export/render time may be high on older devices
 - catalog compatibility rules can become hard to manage without strong metadata tooling
 - App Store monetization UX must be carefully designed
 - sharing expectations may outgrow the first export pipeline quickly
-

Future Build Prompt

Use the prompt below when you want the next system to generate a proper build spec:

Create a production-ready technical specification for an iOS app that lets users build a customizable virtual doll inside a music box. The app should use Swift/SwiftUI on iOS, RealityKit for 3D editing/rendering, AVFoundation for local video export, StoreKit 2 for monetization, Go for the backend API, Redis for cache/job/session state, and PostgreSQL for durable data. The app flow is: animated landing screen, step-by-step doll builder by slots, optional skip for any slot, constrained global adjustment mode, scene decoration, music selection, render short video, share to Instagram via iOS share flow, then payroll after the first successful share/export. Include: system architecture, API design, database schema, Redis usage, asset pipeline for 3D scans, catalog metadata format, project save/load model, purchase entitlement model, analytics events, admin CMS requirements, CI/CD, observability, privacy/security requirements, App Store considerations, phased roadmap, engineering risks, and open product decisions. Optimize for mobile performance and constrained but expressive customization.

Recommended Next Deliverables

The next useful documents to generate from this brief are: 1. PRD 2. technical architecture spec 3. backend API contract 4. database schema draft 5. asset metadata schema 6. iOS screen map and user flow 7. engineering milestone plan